SENG 300 - Intro to Software Engineering

Assignment 1: Project Report

October 11th 2026

Ramsha Oad	30275772
Mawunyo Lotsugah	30242155
Sara Affara	30241537
Chylsy Marable	30241044
Favour Iheanyichukwu	30245516
Nadine Torres	30229594
Parth Singh	30250082

Roles for P1

Name	Phase A&B	Phase C Implementation
Ramsha Oad	- Produce 2 Use Cases for requirements.pdf	Room catalogue FR#1
Mawunyo Lotsugah	- 1 ADR for design.pdf - Interface contracts for design.pdf - 3 project risks for plan.pdf	Draft event management FR#2
Sara Affara	- Produce 2 Use Cases for requirements.pdf	Command line interface
Chylsy Marable	- Produce 2 Use Cases for requirements.pdf - Configure Repo	Student availability FR#2
Favour Iheanyichukwu	- Design UML class diagram for design.pdf	JSON persistence FR#3
Nadine Torres	- Create checklist/Backlog (FR's/NFR's) & Roles for Plan.pdf	Student management FR#1
Parth Singh	- 2 ADRs for design.pdf - 2 project risks for plan.pdf	Shared infrastructure & Seed data

Functional requirements - P1 P2 P3

FR #	Requirement
FR 1	Create, edit, view, and search students. Display the room catalogue and filter by capacity and required features. Validate fields and reject duplicate emails.
FR 2	Enter and update availability. Create, edit, and view draft events with invitees and planning constraints.

	Reject invalid intervals, references, and limits
FR 3	Save and reload students, rooms, availability, and draft events. Preserve ID's and handle missing/corrupt storage. Deliver an interface-to-storage working slice.
FR 4	Generate and explain the top five scheduling suggestions using section 3 rules. Test ranking, boundaries and no-result case.
FR 5	Confirm a suggestion with revalidation, reserve its room, and cancel scheduled events with confirmation. Rejects conflicts without partial changes.
FR 6	Record PENDING, ACCEPTED, DECLINED responses. Enforce participant conflicts and availability rules. Show response and eligible counts separately.
FR 7	List events by date and status and show event details. details. Export one event roster as UTF-8 CSV with header student_id,name,email,response. Quote commas, quotes, and line breaks correctly.
FR 8	Undo and redo successful user mutations within the current session, restoring related data and reservations together. Support at least 20 operations. A new mutation after undo clears redo. Failed operations and exports do not enter history.
FR 9	Implement CR-01: record room closures, identify affected events, and support existing P2 saved files.
FR 10	Implement CR-02: preview and confirm replacement plans, preserve response history, and undo the complete change.
FR 11	Deliver the integrated application, clear error messages, user guide, repeatable build, and documented limitations.

Quality requirements

QR #	Non Functional Requirement
QR 1	Build and tests both run from a clean clone using the documented Maven Wrapper commands.

QR 2	Invalid mutations preserve existing records and reservations.
QR 3	For 20 invitees, 20 rooms, a seven-day window, and 100 existing events, suggestion generation completes within two seconds on the documented test machine over 10 runs, excluding startup.
QR 4	Application operations can be tested without the UI.
QR 5	Persistent data survives a normal restart, and a failed save reports an error without silently discarding the previous valid file.

Schedule for P1

PHASE	GOAL	DUE DATE
a&b	Agree scope, design diagram and plan division of work. Finish	**October 24**
c	Finish implementation and testing	**October 26**

Risks & Retrospective

Risk 1: Components fail to integrate

Different members may implement ConsoleUI, application services and DataStore with incompatible method signatures, error handling or assumptions about mutable data. Individual components could work separately while the complete create-save-reload workflow fails.

Likelihood Medium

Impact High. Integration failures could delay P1 and prevent the required FR-01 to FR-03 demonstration.

Mitigation

- Agree method signatures, request types, return values, exceptions and ownership of mutable data before parallel implementation.
- Integrate one complete student creation and persistence workflow early, using InMemoryDataStore for isolated tests and JsonDataStore for integration tests.

- Run the relevant build and tests before merging changes and review any changes to shared contracts.

Warning signs include repeated signature changes, compilation failures after merges and passing unit tests alongside failed integration tests. If these occur, prioritize the broken shared contract and restore the required P1 workflow before implementing later-release features.

Risk 2: Saved data is corrupted or cannot be restored

A JSON serialization error, invalid stored references or an interrupted write could prevent loading or damage saved records. This would undermine FR-03 and NFR-05 even if the application works correctly before restart.

Likelihood Medium

Impact High. Users may lose access to records, and the restart demonstration may fail.

Mitigation

- Validate loaded data before changing current state, and preserve IDs and references during JSON round trips.
- Write to a temporary file before safely replacing the valid file. Report corruption and save failures without silently overwriting data.
- Test malformed files and simulated write failures, and retain a separate known valid sample dataset for testing.

Warning signs include changed IDs after restart, missing records and load errors after saving. Preserve the affected file for investigation and restore a known valid backup if available. Continue testing with a separate sample file while the defect is corrected; never silently replace the user's damaged file.

Risk 3: Event planning is delayed and blocks other components

UC5 (save and reload) and UC6 (suggestions) both depend on draft events created by EventPlanningService. If this part is late, teammates have no real events to save, reload or build suggestions from, even if their own code is finished.

Likelihood Medium

Impact High. The team could not demonstrate the create, save and reload workflow (FR-01 to FR-03) in P1.

Mitigation

- Push the Event class and an EventPlanningService skeleton with agreed signatures to main early, so others can compile against them.
- Provide hardcoded placeholder events that teammates can use in tests until createDraft works.

- Break FR-02 into small backlog items (create, edit, view, validation) and merge each one as soon as it passes its tests.

Warning signs include no commits, merge requests or finished backlog items for FR-02 a day or two before the deadline. If this occurs, the member alerts the team immediately so the work can be reprioritized or shared, and teammates continue with the placeholder events.

Risk 4: Invalid draft events are accepted

UC4 lists several invalid cases (E1 to E7), such as bad time intervals, unknown student references and exceeded limits. If any case is missed, an invalid draft could be saved and later cause errors in storage or scheduling.

Likelihood Medium

Impact High. FR-02 requires invalid intervals, references and limits to be rejected, and bad data would spread to UC5 and UC6.

Mitigation

- Turn each of E1 to E7 into a checklist item and write one JUnit test per case, using InMemoryDataStore so no UI is needed.
- Validate all inputs before changing any data, and throw a ValidationException or NotFoundException on failure.
- Have a teammate review the validation against UC4 before merging.

Warning signs include a UC4 error case with no test, or a test that passes with invalid input. If this occurs, add the missing test first, then fix the rule, and rerun all tests before merging.

Risk 5: A failed edit leaves a draft partly changed

If editDraft changes some fields and then fails a later check, the draft could be left half-updated. This breaks NFR-02, which requires invalid changes to leave existing records unchanged.

Likelihood Medium

Impact High. Users could lose trust in saved drafts, and damaged data could be saved to the file.

Mitigation

- Validate every new value before changing the draft, so a rejected edit changes nothing.
- Write tests that attempt an invalid edit and then confirm the original draft is exactly the same.
- Only allow edits to events with status DRAFT, and keep all data changes inside the service.

Warning signs include a test where a draft differs after a rejected edit, or users seeing mixed old and new values. If this occurs, stop merging edit changes, fix the order so validation comes first, and add a regression test.

Retrospective

Went well:

- Each feature has its own branch (ex.features/student-management), so unfinished work stays off main.
- Divided tasks well so that each member contributes
- The class diagram gave everyone a shared plan and structure for class names and methods

Could improve:

- Finishing tasks on time we agreed on (tests could not be run without certain classes that were unfinished)
- Constant progress updates so the whole team is on the same page and held accountable
- The team should aim to finish all project requirements a day before the due date to leave room for fixing unexpected issues or implementing features we may have missed.